

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	N. Vinay Yadav	Reg. No.:	192472061
Section / Batch:	CSE – AI / 3 rd Year	Date:	09-10-2026

Code of Conduct

I N. Vinay Yadav (192472061) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

main.py	Visualize	Run ▶	Output
<pre>1 import re 2 email = input("Enter Email: ") 3 password = input("Enter Password: ") 4 mobile = input("Enter Mobile Number: ") 5 email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com org edu net in)\$" 6 password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}\$" 7 mobile_pattern = r"^[6-9]\d{9}\$" 8 if re.fullmatch(email_pattern, email): 9 print("Valid Email") 10 else: 11 print("Invalid Email") 12 if re.fullmatch(password_pattern, password): 13 print("Strong Password") 14 else: 15 print("Weak Password") 16 if re.fullmatch(mobile_pattern, mobile): 17 print("Valid Mobile Number") 18 else: 19 print("Invalid Mobile Number")</pre>			<pre>Enter Email: vinay1234@gmail.com Enter Password: Vinay@1234 Enter Mobile Number: 9874563215 Valid Email Strong Password Valid Mobile Number</pre>

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

Accepted

The screenshot shows a Python IDE with a file named `main.py`. The code defines a DFA simulator with the following logic:

- `transition` dictionary: `q0` transitions to `q1` on 'a' and `q0` on 'b'; `q1` transitions to `q1` on 'a' and `q2` on 'b'; `q2` transitions to `q1` on 'a' and `q0` on 'b'.
- `initial_state` is `q0`.
- `final_states` is `[q2]`.
- `simulate(string)` function: starts at `current_state = initial_state`, tracks the `path`, and returns the path and whether the final state was reached.
- Test strings: `["abaab", "abab", "aab", "abb", "ba"]`.
- For each string, it prints the input string, the transition path, and whether it was accepted or rejected.

The output window shows the results for each input string:

- Input String: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
- Input String: abab
Transition Path: q0 -> q1 -> q2 -> q1 -> q2
Accepted
- Input String: aab
Transition Path: q0 -> q1 -> q1 -> q2
Accepted
- Input String: abb
Transition Path: q0 -> q1 -> q2 -> q0
Rejected
- Input String: ba
Transition Path: q0 -> q0 -> q1
Rejected

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input

Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix

Expected Output

Display all matching patterns based on user selection.

```
main.py Visualize Run ▶
1 import re
2
3 text = """
4 Meeting on 12/09/2026
5 Call 9876543210
6 #NLP
7 #Python
8 @OpenAI
9 @Rahul
10 natural language processing
11 Python programming
12 """
13 print("==== SMART PATTERN MATCHING ENGINE =====")
14 while True:
15     print("\n1. Search Date")
16     print("2. Search Phone Number")
17     print("3. Search Hashtag")
18     print("4. Search Mention")
19     print("5. Search Prefix")
20     print("6. Search Suffix")
21     print("7. Exit")
22     choice = input("Enter your choice: ")
23     if choice == "1":
24         result = re.findall(r"\b\d{2}/\d{2}/\d{4}\b", text)
25
26 main.py Visualize Run ▶
27
28 words = re.findall(r"\b\w+\b", text)
29
30 result = [
31     w for w in words
32     if w.lower().startswith(word.lower())
33 ]
34 print("Prefix Matches:", result)
35
36 elif choice == "6":
37     word = input("Enter suffix: ")
38     words = re.findall(r"\b\w+\b", text)
39
40 result = [
41     w for w in words
42     if w.lower().endswith(word.lower())
43 ]
44 print("Suffix Matches:", result)
45
46 elif choice == "7":
47     print("Exiting program...")
48     break
49 else:
50     print("Invalid choice!")
```

```
Output
==== SMART PATTERN MATCHING ENGINE =====
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 2
Phone Numbers: ['9876543210']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 3
Hashtags: ['#NLP', '#Python']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 2
Phone Numbers: ['9876543210']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 3
Hashtags: ['#NLP', '#Python']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter your choice: 1
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____